In-Storage Computing for Hadoop MapReduce Framework: Challenges and Possibilities

Dongchul Park, Jianguo Wang, and Yang-Suk Kee

Abstract—Solid State Drives (SSDs) were initially developed as faster storage devices intended to replace conventional magnetic Hard Disk Drives (HDDs). However, high computational capabilities enable SSDs to be computing nodes, not just faster storage devices. Such capability is generally called "In-Storage Computing (ISC)". Today's Hadoop MapReduce framework has become a de facto standard for big data processing. This paper explores In-Storage Computing challenges and opportunities for the Hadoop MapReduce framework. For this, we integrate a Hadoop MapReduce system with ISC SSD devices that implement the Hadoop Mapper inside real SSD firmware. This offloads Map tasks from the host MapReduce system to the ISC SSDs. We additionally optimize the host Hadoop system to make the best use of our proposed ISC Hadoop system. Experimental results demonstrate our ISC Hadoop MapReduce system achieves a remarkable performance gain ($2.3\times$ faster) as well as significant energy savings ($11.5\times$ lower) compared to a typical Hadoop MapReduce system. Further, the experiment suggests such ISC augmented systems can provide a very promising computing model in terms of a system scalability.

Index Terms—In-storage computing, in-storage processing, near data processing, smart SSD, Hadoop, MapReduce, big data

1 Introduction

Due to the superior characteristics of Solid State Drives (SSDs) as compared to rotational Hard Disk Drives (HDDs), SSDs have been increasingly adopted in enterprise systems and personal/mobile systems [1], [2], [3]. Originally, SSDs were developed to replace traditional HDDs as faster storage devices. For example, when accessing data, modern SSDs offer over 100 times faster latency than HDDs. Moreover, they consume less than one tenth the energy than HDDs [4], [5]. Thus, most recent research studies have focused primarily on how to exploit SSDs as yet-another-faster HDDs.

However, high computational capabilities of modern SSDs had people start to reconsider SSDs not just a faster storage device, but another type of a computing node, so-called In-Storage Computing (for short, ISC). Unlike a traditional CPU-centric computing philosophy which "moves data closer to computation", the ISC model suggests a totally different computing paradigm-"moving computation closer to data". This offloads key processing functions from host systems to ISC devices, which significantly changes a data flow for computation [6]. This data flow change enables ISC systems to not only achieve faster performance, but marvelous energy saving. It has been reported that reducing data movement can help improve both energy and performance [7]. Further, the energy that data movement consumes is starting to exceed the energy consumed by computation [8]. This ISC paradigm is therefore spotlighted especially in today's big data era [9].

Hadoop MapReduce is a software framework for distributing large data processing on commodity hardware clusters. Its MapReduce framework has became a de facto standard for big data processing [10]. In the MapReduce framework, computation is divided into two functions: Map and Reduce. A Mapper (one element of the Map function) reads a portion of an input data set and generates another data set (i.e., intermediate data) with individual elements consisting of key/value pairs. Reducer (one element of the Reduce function) takes Mapper outputs as input and combines the intermediate data into a smaller set of key/value tuples [11]. The Hadoop MapReduce system tries to collocate data in local computing nodes to exploit data locality, which is at the heart of MapReduce and the reason for its good performance [12]. This Hadoop policy–"putting computation near the data" aligns well with the aforementioned recent computing paradigm shift, which Jim Gray also advocated early [13].

This work explores In-Storage Computing challenges and possibilities for the Hadoop MapReduce framework. We offload Mapper into ISC devices (i.e., SSDs) by implementing Map features inside SSD firmware and integrating our ISC Hadoop framework into the existing Hadoop MapReduce system framework. We also make a deep host Hadoop system analysis to explore a performance bottleneck for system optimization. We configure Hadoop clusters and run a Hadoop MapReduce application on them to verify our holistic ISC Hadoop framework. Our extensive experiments show that the ISC Hadoop system achieves a remarkable performance gain ($2.3\times$ faster) while significantly reducing energy consumption ($11.5\times$ lower). Furthermore, they also verify our ISC Hadoop system scales well—even on low-cost commodity hardware—by noticeably reducing host system computation workloads.

Integrating our ISC Hadoop MapReduce into a system causes several interesting, and very challenging, issues:

- *D. Park and Y.-S. Kee are with Memory Solutions Laboratory (MSL) of Samsung Semiconductor Inc., San Jose, CA 95134. E-mail: {dongchul.p1, yangseok.ki}@samsung.com.*
- *J. Wang is with the Department of Computer Science, University of California, San Diego, CA 92093. E-mail: csjgwang@cs.ucsd.edu.*

Manuscript received 14 Feb. 2016; revised 4 June 2016; accepted 28 June 2016. Date of publication 0 . 0000; date of current version 0 . 0000. Recommended for acceptance by Z. Shao. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below. Digital Object Identifier no. 10.1109/TC.2016.2595566

0018-9340 © 2016 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

IEEE Proof

Data Representation Discrepancy. For data access, host-based Hadoop uses host file systems such as the Hadoop Distributed File System (HDFS) and a native file system such as EXT3/4. However, an ISC device cannot rely on any file system information. Consequently, it uses a different data representation such as a Logical Block Address (LBA). Therefore, a host Hadoop system should be able to collaborate with ISC devices by employing LBA information, not using any file systems.

System Interface Discrepancy. System interfaces between a host system Hadoop framework and an ISC device can be different. That is, a host Hadoop framework uses Java programming language, while our SSD-device ISC framework can adopt a different language like C/C++. Consequently, the host Hadoop system cannot directly communicate with ISC devices.

Data Split. HDFS splits large input data sets into 64 MB chunks (i.e., input split) by default [12]. This HDFS file splitting can cause a serious data split problem in the ISC model. This does not cause any issue in a typical Hadoop system that loads large data in the main memory and processes them in a streaming data access manner. However, since our ISC Hadoop MapReduce framework basically does not move data from devices to the host memory, it inherently cannot avoid this data split issue.

Feature Offloading. We only offload Mapper from a host to an ISC device. Unlike a Map task that does not have a dependency on other tasks, a Reduce task collects the output results of other Map tasks in its shuffle phase. However, since our ISC device cannot directly communicate with other ISC devices, a host system should be in charge of collecting intermediate data from each Mapper. Moreover, the host should redistribute the intermediate data to each ISC device for Reducer execution. This incurs unnecessary redundant data movement and computation overheads.

Pseudo Fully-Distributed Mode. Hadoop basically does not support a fully distributed mode on a single node (i.e., multiple datanode instances on a single machine) because a pseudo distributed mode can configure only one datanode instance. However, the pseudo fully-distributed mode enables to simulate the fully distributed Hadoop clusters with only a single machine. We did a workaround for this mode in order to add as many datanode instances on a single node as needed. All our initial studies are done with this mode.

We address all these challenges in the design section (Section 3) in more detail. The main contributions of this work are as follows:

- *Real SSD implementation*: We implemented Hadoop ISC features (i.e., Mapper) inside SSD firmware. All experiments utilize a real device and reflect authentic system performance.
- *Hadoop system integration and optimization*: We integrated and optimized our ISC Hadoop framework with the existing Hadoop MapReduce framework to support the existing Hadoop features seamlessly.
- *Exploration of challenging issues*: The ISC Hadoop MapReduce framework model causes the aforementioned challenging issues. We judiciously addressed those issues in our proposed ISC Hadoop framework.

The rest of this paper is organized as follows. Section 2 provides an In-Storage Computing and its architecture overview as well as a Hadoop MapReduce framework. Section 3 presents our proposed ISC Hadoop MapReduce framework, and explains our system design and challenges. Section 4 shows our extensive experiment results and provides analyses to demonstrate our ISC Hadoop system performance gain. Section 5 discusses related studies of this work, and Section 6 concludes our work.

2 Background

2.1 In-Storage Computing

In-Storage Computing is a new computing paradigm. Here, a storage device plays a major data processing role as a computing node. Unlike the traditional CPU-centric computing model, available non-CPU components—such as storage devices or Graphical Processing Units (GPUs)—can make a significant contribution to data computation because of their computing capabilities [14], [15], [16], [17].

Though the principal ISC idea has already been proposed and implemented using Hard Disk Drives, it was not widely adopted because of limited HDD computational capabilities [18], [19]. However, modern SSDs have been equipped with significantly stronger computing resources such as multi-core CPUs and abundant DRAM. Hence, people have reconsidered SSDs another type of computing unit, not just faster HDDs.

SSDs typically provide significantly higher internal bandwidth (about $3\times$ higher) than contemporary host I/O interface (SATA or SAS) bandwidths [16], [20]. Thus, moving data from devices to a host results in a significant waste of high internal SSD bandwidth. Low internal I/O latency inside SSDs is another noticeable consideration. A regular host I/O operation is heavily affected by the entire OS stack which introduces extra overheads (e.g., context switches, interrupts, file system overhead, etc.). However, internal SSD I/O latency can avoid those OS software overheads. These factors imply that *I/O intensive applications can significantly benefit from the ISC model* [16], [17].

SSDs are usually equipped with low power processors, not only to save energy consumption but also to lower manufacturing costs. This factor results in ambivalent ISC computing model value. While low power processors (such as many ARM processors) can help ISC applications save energy consumption, their lower processing power (than typical host CPUs) implies *CPU computation intensive applications are not favorable to our current ISC computing model* [16], [17].

2.1.1 ISC Hardware Architecture

Fig. 1 presents the hardware architecture of the modern SSD. The hardware architecture of our ISC SSD device is identical to regular SSDs.

An SSD is largely composed of a NAND flash memory array, an SSD controller, and (device) DRAM. The SSD controller has four main subcomponents: (1) a host interface controller, (2) embedded processors, (3) a DRAM controller, and (4) a flash media controller. The host interface controller processes commands presented to the SSD's host interface(s) (typically SAS, SATA, or PCIe) and distributes them to the embedded processors. The embedded processors receive host commands and pass them to the flash controller. More

IEEE Proof

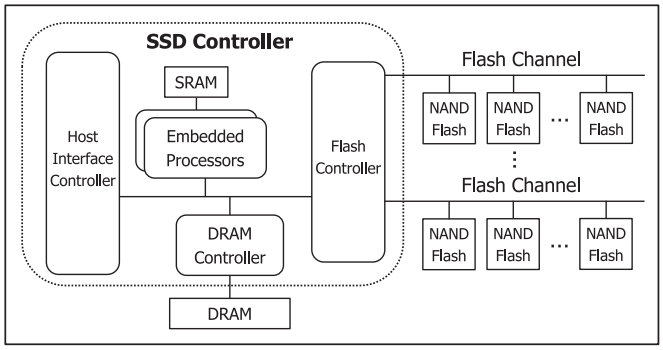


Fig. 1. ISC hardware architecture.

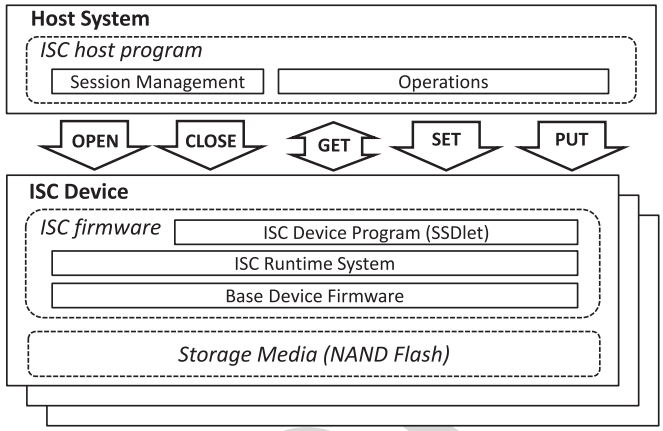


Fig. 2. Our ISC software architecture.

importantly, they run SSD computation firmware and execute Flash Translation Layer (FTL) functions for logical-to-physical address mapping [21], [22]. Typically, they are low-powered 32-bit processors such as ARM processors. Each embedded processor can have a tightly coupled memory (i.e., SRAM) to store performance-critical data or firmware. Each processor can access DRAM through the DRAM controller. The flash controller controls data transfer between flash memory and DRAM.

The NAND flash memory package is the persistent storage media and each package is subdivided further into smaller units that can independently execute commands or report status. An SSD is also equipped with a large memory (DRAM) for buffering data or storing address mapping metadata. All flash channels share DRAM access. Thus, flash channel to/from DRAM data transfers require serialization [16].

2.1.2 ISC Software Architecture

Fig. 2 shows our ISC software architecture composed of two main components: *ISC firmware* inside the SSD and *ISC host program* in the host system. The ISC host program communicates with the ISC firmware through ISC Application Programming Interfaces (APIs).

Since a host system and an ISC storage device cooperatively compute data, we need to define two types of interactions (i.e., ISC host interface and ISC device interface) among a host, a device, and an ISC application. The ISC host interface coordinates communication between the host and ISC devices to control ISC operations. The ISC device interface coordinates internal interactions between the ISC runtime system in the storage device and the ISC application. An ISC host program implements the host interface logic, while an ISC device program (i.e., SSDlet) implements the device interface logic. We implement our ISC applications by using the ISC APIs.

Thus, our ISC programming experiments require us to develop two ISC programs: an ISC host program and an ISC device program called an "SSDlet". We can develop both programs in the host and, depending on the host application, compile the ISC host program with typical C/C++ or Java host program compilers. We also must cross-compile the ISC device program using an ARM compiler to generate the binary code for the device. Next, the SSDlet binary code is downloaded from a host into SSD firmware.[1] After the SSDlet successfully installs in the ISC devices, a host system runs the ISC host program to interact with the SSDlet in the devices.

The ISC firmware has three sub-components: (1) an SSDlet, (2) an ISC runtime system, and (3) base device firmware. An SSDlet is an ISC device program that executes inside an SSD. It implements application logic and responds to the ISC host program in an event-driven manner. The ISC runtime system connects the ISC device program with base device firmware, and implements the ISC API libraries. In addition, base device firmware also implements normal storage device I/O operations (read and write).

This ISC host program largely consists of two components: (1) session management and (2) an operation. The session component manages the ISC device application lifetime for a session. Here, the ISC host program launches an SSDlet by opening a session to the ISC device. To support this session management, our ISC programming model provides two APIs, namely, (1) OPEN and (2) CLOSE. OPEN starts a session and CLOSE terminates a session. Once OPEN starts a session, runtime resources such as memory and threads are assigned to run the SSDlet and a unique session ID is returned to the ISC host program. When CLOSE terminates an established session, it releases all assigned session resources and closes the SSDlet associated with the session ID.

Once OPEN establishes a session, the operation component helps the ISC host program interact with an ISC device SSDlet with GET, SET, and PUT APIs. The GET operation checks SSDlet status and receives SSDlet output results if results are ready. The GET API implements the SAS/SATA interface polling mechanism because, unlike PCIe, such traditional block devices cannot initiate a host request such as an interrupt.

Aside from the existing OPEN, GET, and CLOSE APIs, we implemented two additional new APIs: SET and PUT. The SET operation delivers new ISC metadata information (e.g., starting LBA, data length, etc.) from a host to an ISC device within a session. The SET API is very useful to process multiple non-contiguous data within one session. Without the SET API, it was necessary to close the existing session and re-open another, causing unnecessary overhead. PUT internally writes data to the NAND flash media without host file system assistance. This PUT API is useful to store intermediate ISC processing data.

1. We can use commercial software such as LSI DriveBadger or company proprietary software to download the device binary code into SSD firmware.

IEEE Proof

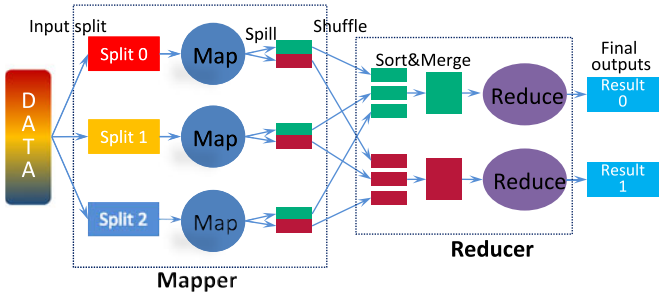


Fig. 3. Hadoop MapReduce framework.

2.2 Apache Hadoop

Hadoop is an open source implementation of the original MapReduce parallel programming model and system designed by Google [11]. Hadoop provides a distributed file system (i.e., Hadoop Distributed File System) that reliably stores very large data sets and a framework (i.e., MapReduce) to process/analyze the data sets. An important Hadoop characteristic is that it partitions data and computation across many hosts, and the parallel application computations are close to their data. Therefore Hadoop clusters easily scale computation capacity, storage capacity and I/O bandwidth by simply adding commodity servers and storage.

2.2.1 Hadoop Distributed File System

The Hadoop Distributed File System is an open source implementation of Google Distributed File System (GFS) [23]. By design, HDFS is heavily optimized for manipulating very large files with streaming data access patterns (i.e., write-once, read-many patterns).

HDFS stores file system metadata and application data separately. That is, HDFS stores metadata on a dedicated server called the namenode. Application data are stored on other servers called datanodes. All servers fully connect and communicate with each other using TCP-based protocols. HDFS clients perform file system metadata operations through the namenode and send/retrieve file system data by communicating with a pool of datanodes. For reliability, datanodes usually does not rely on data protection mechanisms such as RAID. Instead, the application data are replicated on multiple datanodes (typically three replicas by default). Thus, the loss or failure of a single datanode is never fatal to the Hadoop cluster and never causes data loss. Though this HDFS data replication strategy can incur network bandwidth overheads, it increases data locality opportunity by locating computation near the needed data and this meets the aforementioned Hadoop philosophy: putting computation near the data.

2.2.2 Hadoop MapReduce Framework

MapReduce is a software framework for distributed processing of large data sets on commodity clusters [24]. The Hadoop MapReduce framework leverages the Hadoop Distributed File System. A MapReduce job consists of both Map and Reduce tasks, and one complete MapReduce job essentially has three phases: (1) Map, (2) Shuffle, and (3) Reduce phases.

As illustrated in Fig. 3, when Hadoop MapReduce processing begins, Hadoop splits large input data into predefined-sized data chunks (by default 64 MB). Each data split is fed to a single Map task. The collective Map tasks execute

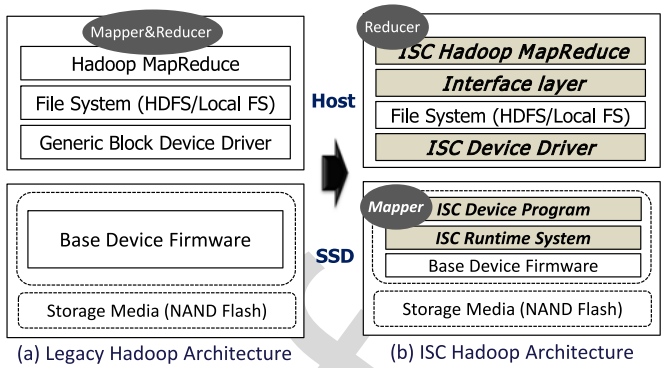


Fig. 4. ISC Hadoop MapReduce architecture.

in parallel on many physical machines. A Map task applies the Map function to its own input data split and generates an output data set (i.e., intermediate key-value pairs). If a user implements an optional combine function, a Map tasks then performs this combine function to its key-value pair list. Next, each Map task partitions the intermediate data (key-value pairs) into the number of Reduce groups. We offload this Map function into the ISC device.

Once all Map tasks successfully completed, the Hadoop MapReduce framework transfers the Map outputs to the Reducers as inputs, a process known as the Shuffle. Each Reduce task requests one intermediate data file from each Map task and starts to merge them based on intermediate keys. Sometimes this Shuffle phase is considered a part of Reduce phase and called a Reduce phase 'copy' step.

After all Reduce tasks copy their intermediate data, they start to sort and merge them. Then, they apply a Reduce function to the sorted data and emit the final output for each Reducer. Reduce tasks write their final output to a file using HDFS.

3 ISC Hadoop MapReduce

3.1 System Architecture

Fig. 4 shows the overall system architecture of our ISC Hadoop MapReduce framework. As stated, we offload the Mapper into an ISC device and implement Mapper features inside the SSD firmware. Thus, a host Hadoop MapReduce system must be modified to interoperate with our ISC device. This requires disabling host system Mapper features and implementing them inside SSD firmware (i.e., ISC device program). To support ISC functions, we also need to develop ISC engines (i.e., ISC Runtime System) on top of existing base device firmware. The interface layer plays the role of an agent, communicating between the modified host Hadoop system and our ISC device. We suggest this software component to resolve two major design issues: (1) system interface discrepancies and (2) data representation discrepancy between the host and the ISC devices (these issues will be addressed in detail in Sections 3.3.1 and 3.3.2.). Alongside of a generic SCSI block device driver, we implement a special SCSI command to trigger ISC features inside the SSD (ISC Device Driver). We do not touch either HDFS or the local file system. So, our ISC Hadoop MapReduce system seamlessly supports all existing Hadoop features.

We first explore our design on a single Hadoop node and then extend it to multiple-node Hadoop clusters.

IEEE Proof

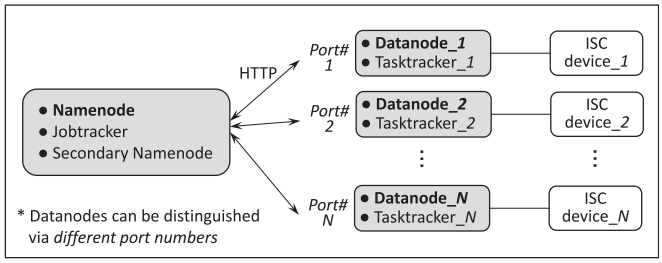


Fig. 5. ISC Hadoop configuration on a single node: a pseudo fully-distributed mode on a single machine.

3.1.1 A Single Node: A Pseudo Fully-Distributed Mode

There are three typical ways to install and configure Hadoop systems: (1) a local (standalone) mode, (2) a pseudo distributed mode, and (3) a fully distributed mode. In the standalone mode, all Hadoop components (i.e., namenode, jobtracker, datanode, tasktracker, secondary namenode) run in a single Java process (i.e., JVM instance). This mode does not use HDFS. In the pseudo distributed mode, Hadoop runs all components as separate Java processes and uses HDFS. This simulates fully distributed Hadoop clusters but only runs *a single instance of datanode on a single machine*. The fully distributed mode runs on multiple-machine clusters with a master (namenode, job tracker, and optional secondary namenode) and slaves (datanodes and tasktrackers) [10].

In addition to these three existing modes, we suggest another mode: a pseudo fully-distributed mode. As stated, it is basically impossible to run Hadoop in a fully distributed mode on one local machine (i.e., *multiple instances of datanode on a single machine*). However, this workaround mode serves as a fully distributed Hadoop cluster on a single node. Fig. 5 illustrates this Hadoop configuration on a single node. We configure separate Hadoop configuration files for each datanode instance. For HTTP communication between the namenode and datanodes, we assign different port numbers to each datanode instance, wherein each datanode connects with our ISC device. Thus, each datanode can be distinguished via different communication port numbers with the datanode.[2] With this mode, we can add as many datanode instances on a single node as needed.

This pseudo fully-distributed mode has the following merits. First, we can efficiently simulate distributed Hadoop clusters by using a single machine. Consequently, we can effectively verify real distributed Hadoop clusters with less effort (e.g., debugging, time sync, cost, etc. in real distributed systems). Second, like Reducer, we can precisely control the number of Mappers running on the Hadoop system. In the typical Hadoop system, we can configure only the *maximum* number of Mapper in the node and the Hadoop system automatically adjusts the number of Mappers based on the system resources (e.g., CPU cores). On the other hand, this new mode enables us to control the specific number of Mappers running on the single Hadoop system. Consequently, we can efficiently verify ISC system effectiveness in terms of scalability by adding many ISC devices (up to eight in our experiments) into the node. Most of our experiments and design are based on this single node configuration.

2. This configuration is based on Hadoop 0.20.205.0 and we found the assignment of virtual IPs to each datanode instance did not work for this workaround in this Hadoop version.

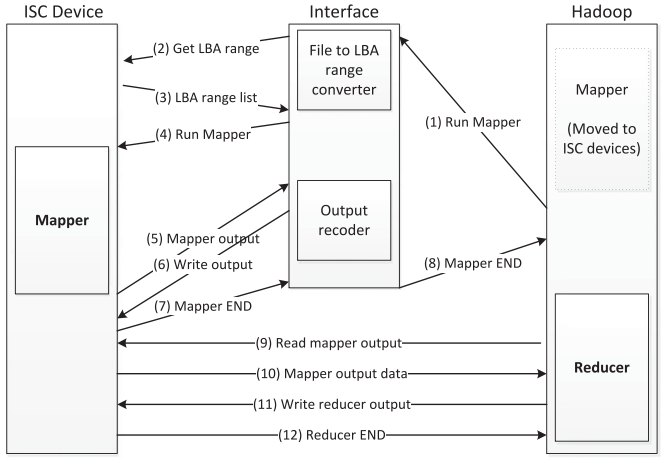


Fig. 6. ISC Hadoop MapReduce workflow.

3.1.2 Clusters: Multiple Nodes

The aforementioned single node configuration represents the main features of the distributed Hadoop clusters on a local machine very well. Based on this, we extend our ISC Hadoop MapReduce framework to real Hadoop clusters. We set up five node clusters and separate a namenode from other datanodes (i.e., one namenode and four datanodes) to completely eliminate its performance overheads from the datanode. We initially start our study with one datanode (i.e., one namenode and one datanode) and gradually increase the number of datanode to four on our clusters. Since we found meaningful and predictable performance results (that is, almost linear performance increases over the number of datanodes), we did not configure bigger Hadoop clusters (please refer to our experiment results).

3.2 Workflow

Fig. 6 presents our ISC Hadoop MapReduce workflow. Once a Hadoop application runs, our ISC Hadoop MapReduce framework does not directly run Mappers on the host system because they moved inside ISC devices. Instead, it first calls an interface component to communicate with ISC devices while delivering the input data split information (i.e., a file name of the input split) for ISC Map tasks. The interface runs its software component for converting each input data split (64 MB) information to different data representation (i.e., LBA range information) for ISC devices. We found each input 64 MB data split is generally further split into 2 or 3 smaller data chunks via the host file system, EXT4 in our case. This means the interface software component (i.e., File-to-LBA range converter) delivers 2 or 3 LBA range lists to an ISC device for each Map task. Restated, the interface component calls Mapper inside the ISC device with these converted LBA range lists. After the Mapper receives all metadata information from the interface layer, it starts executing a Map task. Once the Map task completes, an ISC device returns its Mapper outputs (i.e., intermediate results) to the interface layer to save in the device (that is, a Mapper output is written with a local file system). In fact,

IEEE Proof

these Map output writing processes (Steps 5 and 6) can be replaced with our new ISC API–PUT. Now, host Reducers can start to read the intermediate data as their input values and perform Reduce tasks. Finally, once all Map and Reduce tasks complete, our Hadoop system stores the final results using HDFS.

3.3 Design Challenges

3.3.1 Data Representation Discrepancy

To access data stored in a device, a host system can utilize file system information. So does the host Hadoop system by employing HDFS or local file systems such as EXT3/4. However, the device itself cannot rely on any file system to access data inside its own device for ISC processing. This data representation discrepancy between the host and the device causes an important problem. This is a primary reason previous ISC work [16], [25] pre-defined their data locations to the raw device with a 'dd' command, explicitly specifying their data locations (LBAs) from the beginning. This approach may be able to avoid this issue temporarily, but cannot resolve it. Consequently, it gave their systems other critical limitations. For instance, Kang et al. proposed a Hadoop MapReduce system leveraging ISC [25]. However, due to this challenging issue, their Hadoop system utilizes only one local machine with only one ISC device (i.e., no scalability) since it cannot support HDFS and local file systems.

We resolved this critical issue by developing a software component (i.e., file-to-LBA range converter) in the interface layer between the Hadoop system and ISC devices (please refer to the Fig. 6). We used FIBMAP with ioctl Linux API to get a block layout of files (i.e., HDFS data splits of 64 MB) and FIGETBSZ to get the total number of blocks the files occupy. Since the HDFS data files are large (by default 64 MB), they are highly likely to be fragmented in the EXT3/4 file system (based on our observation, mostly 2-4 fragments). Therefore, the software component converts each 64 MB data file into multiple (mostly 2-4) LBA range lists and then delivers the LBA list information to an ISC device for Map task processing. Consequently, our proposed ISC Hadoop MapReduce system seamlessly supports existing Hadoop features.

3.3.2 System Interface Discrepancy

System interfaces between the host system Hadoop framework and an ISC device can be different. That is, the host Hadoop MapReduce framework adopts the Java programming language, while our ISC processing firmware uses the C/C++ language. Consequently, host Hadoop system cannot communicate directly with ISC devices. To resolve this issue, we can think of two options: (1) running an OS inside ISC devices and (2) adopting an interface layer. If an OS runs inside ISC devices, it can also simply run JVM on top of the OS. This solution looks like very intuitive and may be able to eliminate our aforementioned challenge (i.e., data representation discrepancy) as well. However, this approach consumes extra ISC computing resources such as CPU and DRAM inside the device. Thus, unless the ISC device is a very specialized device with powerful computing resources, the first option is not a good choice.

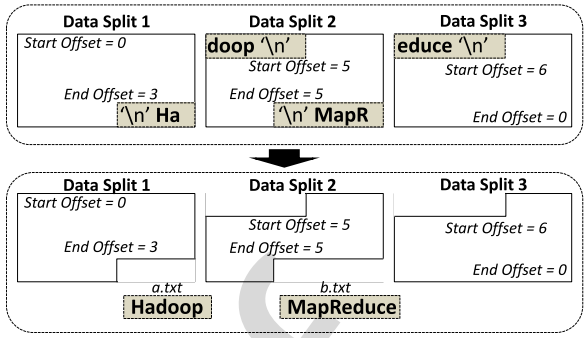


Fig. 7. A data split example.

Since our ISC device is a typical commodity SSD without sufficient extra computing resources to run an OS, we choose the second option. Instead, we put another interface layer between the Hadoop system and ISC devices by adopting the Java Native Interface (JNI). Aside from the programming system interface between them, this interface layer performs two important ISC Hadoop System roles: (1) as mentioned above, converting a file name to LBA range information, and (2), writing ISC Map task output data (i.e., intermediate data). For this, the host Hadoop system must deliver metadata information including an input split file name and an output file name for the intermediate data. The overhead of this extra interface layer is nearly negligible.

3.3.3 Data Split

Hadoop Distributed File System splits large input data into 64 MB chunks by default [12]. This input data split process raises another critical ISC Hadoop system problem: data splitting. Fig. 7 illustrates this issue. Assuming the word 'Hadoop' is stored in the large input data, HDFS may split the word 'Hadoop' into 'Ha' in one file of 64 MB (i.e., Data Split 1) and 'doop' in another file of 64 MB (i.e., Data Split 2) if the word inadvertently lies on the input split boundary. This does not cause an issue in a typical Hadoop system because the host Hadoop system HDFS loads all input data splits into main memory and processes them as streaming data. However, since ISC Hadoop MapReduce framework basically does not move input data from devices to host memory, this data split issue can cause a critical problem in an ISC Hadoop system. If ISC devices are equipped with a capability that allows them to directly connect/communicate with each other like Ethernet-connected drives (e.g., Seagate's Kinetic HDD or HGST's Ethernet Drive [26], [27]), we may be able to solve this issue with a naive way. But, unfortunately, almost commodity SSDs are not equipped with such a communication interface and neither is our ISC drive.

To resolve this, our ISC Hadoop system employs the host Hadoop system. If the host Hadoop system detects a data split by checking the last byte of each data file,[3] it seeks forward/backward to find the predefined delimiters such as a space or tab. Once it finds them, it concatenates both terms into one word and stores it (both Hadoop and

3. The host Hadoop system originally has a simple algorithm for this data split detection.

IEEE Proof

MapReduce in the simple example) as a separate file with a local file system. Then it converts the file name to LBA information by using the aforementioned software component and delivers the LBA information to the ISC device via the interface layer. To avoid duplicate computation of that part inside ISC device, the host system provides the ISC device with other metadata information (that is, Start Offset and End Offset) for each data split. These offsets can be regarded as each part's data length including a delimiter, where the delimiter can be a carriage return, space, or tab, depending on the Hadoop applications. With the help of this metadata information, an ISC Mapper can simply ignore processing all data within the offset boundary in each data split chunk of 64 MB.

3.3.4 Feature Offloading

Hadoop MapReduce framework processing is composed of Map and Reduce. Unlike a Map task that does not have any dependency on other tasks for execution, a Reduce task is inherently dependent on other Map task executions. That is, a Reducer is required to collect output results of other Map tasks in its shuffle phase before a Reduce task starts. However, as mentioned in the previous Section 3.3.3 (Data Split), our ISC device is not equipped with such a stand-alone communication capability. Moreover, a host system should be in charge of collecting the intermediate data from each Mapper and should redistribute the collected data to each ISC device for Reduce processing. Not only can these processes incur unnecessary data movement traffic, but they can overwhelm ISC devices because an ISC device typically has very limited resources and both Mapper and Reducer generally concurrently execute.

Many Hadoop MapReduce applications (not all, but most) produce small amount of intermediate data after a Map task processes original big data. In addition, the Reduce task has three major phases such as copy, sort, and reduce. Unlike a Map task, the Reduce task generally does not significantly reduce the data size of its output result after Reduce processing. This is not favorable to an ISC framework [16]. The sort can give rise to another unfavorable issue. Assuming a data size is bigger than available ISC device DRAM space, it may have to issue numerous NAND flash Write operations to store intermediate results, which critically impacts NAND flash memory life endurance [28], [29], [30], [31].

Based on these observations, we move Mapper to ISC devices, but not Reducer.

3.4 ISC Hadoop Optimization

While conducting our ISC Hadoop system experiments, we found an interesting result: even though the pure ISC Hadoop system Mapper execution time dramatically decreases from 12.7 to 4.7 seconds, it does not correctly reflect each Map task completion time. That is, each Mapper always takes 6 seconds to complete its task even though it completes the Map task very quickly (i.e., less than 6 seconds). This implies the typical Hadoop system does not make the best use of our ISC Hadoop system. This is a primary motivation for Hadoop optimization of our proposed system. We not only tried to apply most widely-known Hadoop optimization approaches [32], but also thoroughly investigated the time required for all Hadoop task execution phases to identify other performance bottlenecks. We particularly suggest the following major Hadoop optimization for our ISC Hadoop MapReduce framework.

Progress Interval: The communication thread in each task handles tasktracker communication. It sends progress updates if progress has been made or also pings the tasktracker to see if it is alive at 3 second intervals. Because of this coarse progress interval, each Map task completion time always appears to require 6 seconds even though its execution completes in less than 6 seconds. Interestingly, even when it completes in less than 3 seconds, we found it still appears to require 6 seconds, implying this becomes the most critical bottleneck if we adopt a faster ISC device in the future. Therefore, we must adjust this progress interval to a finer-grained value in the codes.

Tasktracker Out-of-Band Heartbeat: This is a well-known Hadoop optimization approach [32]. If we enable this configuration, Hadoop tasktracker sends an out-of-band heartbeat when the task completes in order to reduce latency. We verified this also notably helps improve performance.

Heartbeat Interval: At this interval (3 second default), Jobtracker polls tasktrackers to check their task completion and to schedule other tasks to each tasktracker. By reducing this value to a finer-grained interval, we increased scheduling efficiency. As a result, it improves overall Hadoop performance.

TABLE 1
Parameter Setup

Property	Configurations
Cluster	CPU: Intel Core i5-4570 (3.2 GHz), 4 cores Memory: 8 GB DRAM (DDR3-1600) Node #1 runs namenode and jobtracker Node #2-#5 run datanode and tasktracker All nodes connected via 1 Gb Ethernet
Server	CPU: Dual Intel Xeon E5-2630 (2.3 GHz), 6 cores each (total 12 cores) Memory: 32 GB DRAM (DDR3-1600), ECC Reg.
Storage	Samsung SM1625 SSDs (400 GB, SLC, 6 Gb SAS)
Software	Native Hadoop 0.20.205.0 Ubuntu 12.04 LTS, 64 bit OpenJDK 1.6.0_27, 64 bit
Hadoop	Replication factor: 3 Number of Reducer: 2 Number of Mapper: 1 per ISC device HDFS file size: 64 MB
Data size	1 GB, 2 GB, 4 GB, 8 GB, 16 GB

4 Experimental Results

4.1 Evaluation Setup

4.1.1 System Configurations

Table 1 describes our system configurations and parameters. We adopt Hadoop 0.20.205.0 as our Hadoop MapReduce framework because this has been widely adopted as a stable legacy version and use an HDFS size of 64 MB by default.

For a single node setup, we prepare two different nodes to explore ISC scalability: (1) a low cost commodity desktop with an Intel i5-4570 processor (3.20 GHz, 4 cores) and 8 GB

IEEE PROOF

memory, and (2) a server with higher computing power (dual Intel Xeon E5-2630 processors with 12 total cores and 32 GB memory). Both have a 400 GB SLC SSD. The SSD (i.e., ISC device) has a 6 Gb SAS interface and a 400 MHz ARM Cortex-R4 processor. This ISC device connects to the host via a 6 Gb SAS Host Bus Adapter (HBA). An ISC device has identical hardware specifications and base firmware compared to a regular SSD. We added ISC features by extending the existing SSD base firmware. Thus, if a host program sends ISC commands to the SSD via ISC APIs, the SSD operates as an ISC device. Otherwise, it operates as a typical SSD. All nodes run Ubuntu 12.04 LTS (64 bits).

We first install one ISC device and gradually increase the number of ISC device up to eight devices in the single node to explore scalability. This single machine configuration simulates multiple node Hadoop clusters for a fully distributed mode. For instance, a single node with eight ISC devices corresponds to Hadoop clusters with one namenode and eight datanodes. We assign one Mapper to each ISC device and configure the host Hadoop system with two Reducers.

A Hadoop cluster consists of five nodes (one namenode and four datanodes) with aforementioned business desktop PCs. We initially start our study with one datanode (i.e., one namenode and one datanode) and gradually increase the number of datanode to four on our clusters. Since predictable performance results were observed (that is, almost linear performance increases over the number of datanode), we did not configure bigger Hadoop clusters. For fairness, one Mapper is assigned to each ISC device and one ISC device is installed to each datanode. Each node connects via a 1 Gb Ethernet switch.

As our baseline system, we adopt a typical Hadoop MapReduce system with SSDs running all Mappers and Reducers on the host side. That is, for fairness, we employee exactly the same system as the ISC Hadoop MapReduce system described above except for disabling ISC features inside our SSDs.

We employ various synthetic data which vary in size from 1 to 16 GB. All our experiments run 30 times for statistical significance and error bars are not displayed in the figures because standard deviation values are almost negligible.

4.1.2 Performance Metrics

We mainly measure an elapsed time (seconds) as our Hadoop system performance metric and collect all execution time information from Hadoop JobHistoryServer, thereby using *'hadoop job -history'* command.

First, we measure an end-to-end elapsed time. This corresponds to a Hadoop application total execution time and includes all setup, Map, Shuffle, Reduce, and cleanup time. This performance represents an actual application performance where end users can be conscious of the system performance gain. For deep analysis, we delve into Mapper because our ISC Hadoop framework offloads Mapper to the ISC device and the performance gain primarily originates from the Mapper inside our ISC devices.

Energy consumption is also another important factor. We measure the energy consumption (in joules) with a Yokogawa WT330 power meter [33] as follows. Once all systems sufficiently stabilize (i.e., are idle), we measure the power in Watts (W_1). Then, while running the systems, we measure the power (assuming W_2) during execution (t). Now energy consumption in Joules can be drawn by $(W_2 - W_1) \times t$.

Finally, we measure host system resource usage (CPU in percentage, DRAM in GB, and I/O in MB/sec) to analyze our ISC Hadoop system performance gain.

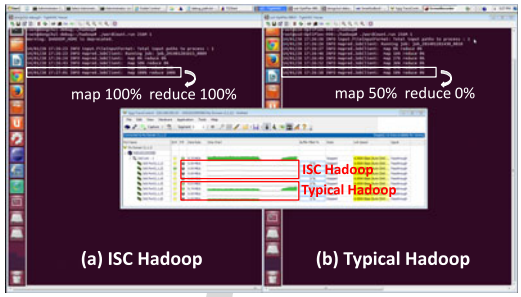


Fig. 8. Hadoop system demonstration (ISC Hadoop versus typical Hadoop).

4.1.3 Hadoop Applications

Admittedly, not all Hadoop applications benefit from our ISC Hadoop MapReduce framework. As Section 2.1 and previous work [16], [17], [25] discuss, our ISC model is best utilized by applications having the following characteristics: (1) I/O intensive, (2) high data selectivity (i.e., less data access per flash page), and (3) a small amount of output data after processing. We can find many I/O intensive Hadoop applications producing a small amount of final outputs. Unfortunately, Hadoop applications cannot meet the high data selectivity since they must read every data byte for processing.

For Hadoop application study, we refer to Purdue MapReduce (PUMA) Benchmarks Suite [34]. It presents a broad range of Hadoop MapReduce applications (13 applications) exhibiting various characteristics. After the investigation, we found many of them can be classified as "wordcount-like" applications. That is, (1) Mapper intensive (i.e., average per-Mapper execution time is significantly longer than the average per-Reducer execution time), (2) moderately CPU intensive computation, (3) counting-related output values, and (4) significantly smaller output data amounts.

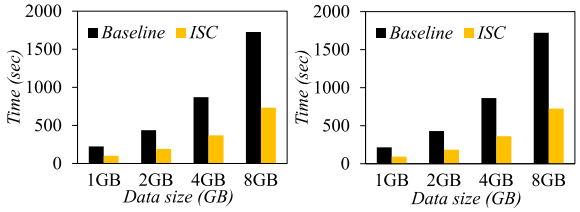
Based on our study, we choose Hadoop wordcount as our ISC application. Please note the main goal of this work is not to run various Hadoop applications, but to verify the correctness of our ISC Hadoop MapReduce framework and to explore its possibilities and challenges as a pioneering work. Therefore, we leave implementing/running various applications on our ISC framework, even ISC-unfavorable ones, as future work.

4.2 Evaluation Results

4.2.1 ISC Hadoop Demonstration

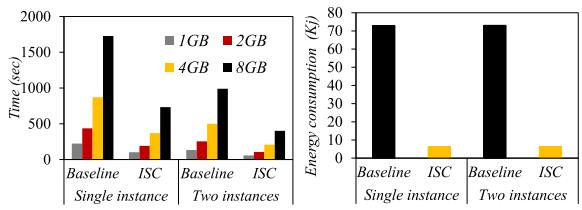
We developed a Hadoop demonstration system that compared our ISC Hadoop system with a typical Hadoop system. Fig. 8 depicts this demonstration system. We set up both Hadoop systems on a single node with a two datanode and one namenode configuration, and ran the Hadoop wordcount application with the same data at the same time. Both systems connected to a SAS/SATA bus analyzer [35] to observe the amount of data transfer from devices to a host system (green histogram). As Fig. 8 shows, our ISC Hadoop system completes tasks faster than a typical Hadoop system

IEEE Proof



(a) Total Elapsed Time (b) Total Mapper Elapsed Time

Fig. 9. Total elapsed time on a single node (single datanode and single namenode).



(a) Total Elapsed Time (b) Total Energy Consumption

Fig. 10. Single instance (1 ISC device) versus two instances (2 ISC devices) on a single node.

(i.e., when the ISC Hadoop system already completes both all Map tasks and Reduce tasks, the typical Hadoop system completes only 50 percent of Map tasks and does not even start Reduce tasks.). Moreover, we clearly observe the ISC system does not (or very rarely) transfer data from the device to the host (upper versus lower red rectangle). Note the uppermost green histogram represents a total amount of data for both ISC and typical Hadoop systems. Now, we provide various analyses of this result via extensive experiments.

4.2.2 A Single Node

We set up a Hadoop system for the pseudo fully-distributed mode on a single node with a variety number of ISC devices and perform extensive experiments.

A Total Elapsed Time. Fig. 9 shows the total elapsed time on a single node with a single namenode and datanode. Our ISC Hadoop system completes 2.3× faster than the typical Hadoop system in terms of a total elapsed time (i.e. end-to-end total execution time) (please refer to Fig. 9a). Fig. 9b displays a total elapsed time in a Mapper phase by excluding all other phases. This shows how much time each Hadoop system consumes to complete all Map tasks. Similarly, our ISC Hadoop system completes all Map tasks 2.3× faster than the baseline system. The results and trends in Fig. 9b are almost identical to those in Fig. 9a. This is because the total execution time of all Map tasks is a significant dominant factor of the total Hadoop running time in wordcount-like Hadoop applications (for instance, 91 percent with 1 GB data—98 percent with 8 GB data of the total wordcount execution time).

Fig. 11a demonstrates this analysis very well. It breaks down the total execution time of wordcount into each Hadoop phase (baseline with 1 GB data on a single node). After a short Hadoop setup time (1 second), the Hadoop system keeps executing each Map task (216 seconds) and the completed Map tasks trigger the parallel execution of a corresponding Reduce task with their output data. The total execution time of Reduce tasks (208 seconds) almost overlaps with that of Map task since it is heavily dependent on Map task completion. Lastly, Hadoop cleanup time takes about 1 second.

Fig. 11b makes a deeper analysis of the Hadoop task execution time. This figure shows the elapsed time of each Hadoop phase for each task. Each Hadoop task is subdivided into Map task and Reduce task. Hadoop sets up a job (setup time) at the beginning and deletes all intermediate data (cleanup time) after job completion. These two tasks are respectively performed only one time throughout the whole job execution. That is, the total execution time of Hadoop application can be largely consist of a one-time job setup time, many (depending on a data size) Map and Reduce task execution times, and a one-time cleanup time. Considering this, Mapper execution time is a dominant factor for total Hadoop execution time in wordcount-like Hadoop applications. Based on these observations, our proposed ISC Hadoop framework moves Mapper to our ISC devices to improve Hadoop performance. As shown in Fig. 11b, our ISC Hadoop system improves Map task execution time by 2.6× (from 13 to 5 seconds). All performance gains stem from the Mapper performance improvement.

Fig. 11c provides an even deeper analysis of the Mapper execution time. This plot corresponds to Map task execution time (i.e., yellow bar) in the Fig. 11b. Similarly, Map task execution time is further subdivided into three elements: (1) Map task setup time, (2) Map task execution time, and (3) Map task cleanup time. Both Map task setup time and cleanup time are almost negligible (less than 0.1 seconds in total). Thus, if we compare a *pure* Map task execution time (i.e., yellow bar in the Fig. 11c) between them, our performance gain is a little larger (2.7×).

Now, we add one more ISC device to the host, configuring a single namenode instance and two datanode instances on a single machine. Fig. 10a clearly indicates each system

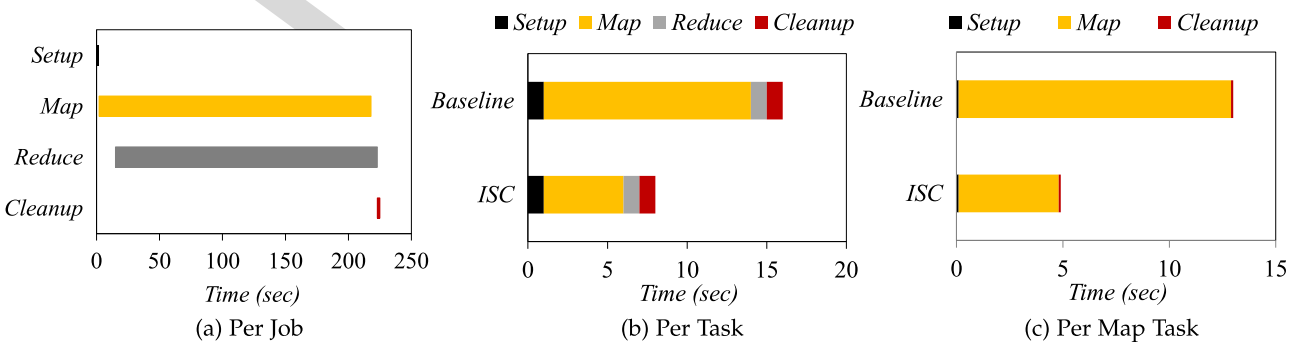
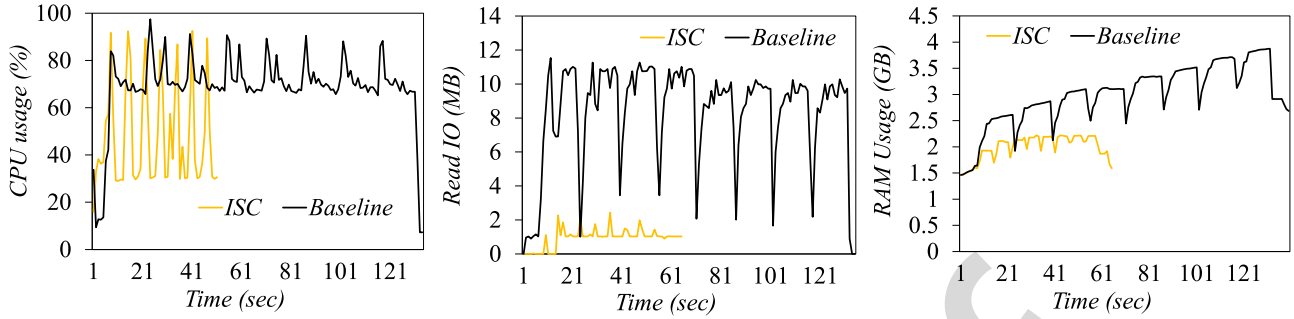


Fig. 11. Breakdown of each Hadoop execution time.

IEEE Proof



(a) Host CPU Usage (b) Host Read I/O (c) Host DRAM Usage

Fig. 12. Host system resource usage.

achieves 2× performance improvement with this configuration compared to the system with a single ISC device. This experiment result provides a meaningful implication that our ISC Hadoop system, even with its lower system cost can offer a better performance than a typical Hadoop system. We increase the number of ISC devices up to eight devices on a single machine and this is addressed in the scalability evaluation section.

Energy Consumption. Fig. 10b shows total energy consumption of each Hadoop system. We run the Hadoop application with 10 GB data on both systems and measure energy consumption. As stated in Section 4.1.2, we subtract the idle power (40.5 W for the single instance, 45.5 W for the two instances configuration) from the total power consumption. Then, it is multiplied by the total execution in order to draw the total energy consumption. As shown in Fig. 10b, our ISC Hadoop system consumes surprisingly lower energy (11.5×) than the typical Hadoop system with SSDs. Since we measure an additional energy consumption by eliminating an idle system power, both Hadoop systems with single ISC devices (one SSD for the baseline system) and two ISC devices (two SSDs for the baseline system) show almost the same results. Intuitively, this is because the Hadoop system with two ISC devices consumes two times more system power (after subtracting the idle power) but it still completes Hadoop job two times faster. Based on our experimental results, with the help of a faster execution time and lower energy consumption, our ISC Hadoop system can make a notable contribution to reduce Total Cost of Ownership (TCO).

4.2.3 A System Resource Usage

All key ISC system benefits originate from the fact it does not move storage device data to host system DRAM. This section verifies this claim.

We capture three main system resources of a host: Host CPU usage, Read I/O, and DRAM usage. For this, we set up two Hadoop systems (i.e., ISC versus baseline) respectively on a single machine with two SSDs (or ISC devices) and run a Hadoop application with 1 GB data. We observe each system resource over the execution time (Fig. 12). Interestingly, we observe 8 repetitive ridges in the Fig. 12. This is because we use 1 GB data and configure two datanode instances on a single machine where one Mapper is assigned to each instance. Each datanode executes its own Map task by loading its respective 64 MB HDFS data split file (that is, 128 MB for two simultaneous Map tasks). Therefore, we can see eight separate sections (1 GB = 128 MB × 8).

A Host CPU Usage. Fig. 12a shows the host CPU usage during Hadoop application execution time. This demonstrates our ISC Hadoop system consumes even less host CPU resources (3.7× less on average and 4.2× less for Map task execution periods). This is because the ISC Hadoop system executes all Map tasks inside ISC devices, thereby using their own CPUs and does not (or very rarely) transfer data to the host system for Map task computation.

A Host Read I/O. Unlike the conventional CPU-centric system loading all data to the host for computation, an ISC system does not need to move data from a device to the host. This significantly reduces Host I/O between them. Fig. 12b supports this claim. A typical Hadoop system generates lots of read I/Os to load all data from devices to the host, while our ISC Hadoop system very rarely generates read I/Os (almost two orders of magnitude less). Especially for the period of each Map task computation, the typical Hadoop system keeps reading data from the devices. This is also verified by our SATA/SAS bus analyzer (Fig. 8). We capture both read and write I/Os, but write I/Os are almost ignorable (less than 1 percent) for this application. Thus, we exhibit only read I/Os in the plot.

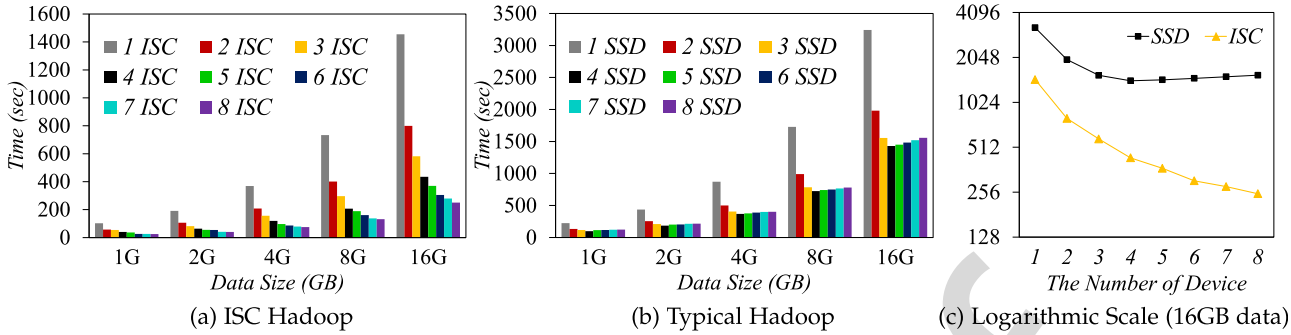
A Host DRAM Usage. Host DRAM consumption is another important factor in favor of ISC systems. Typically, all data loaded from devices are first stored in host DRAM for computation, increasing memory consumption. To measure this host memory consumption, we do not run other applications after reboot and flush all data/page caches. After running the Hadoop application, the baseline system keeps consuming DRAM until all Map tasks complete. It follows that our ISC Hadoop system consumes less DRAM (3× less). This has a critical energy consumption impact since it has been widely known that DRAM takes 25-40 percent of a system power consumption [36], [37], [38], [39].

4.2.4 Scalability

Scalability is a primary force driving Hadoop MapReduce framework popularity and adoption in big data processing. In-Storage Computing is another promising model for system scalability because of its significant host system resource savings (as addressed in Section 4.2.3). Now, we observe how effectively ISC can help scale systems.

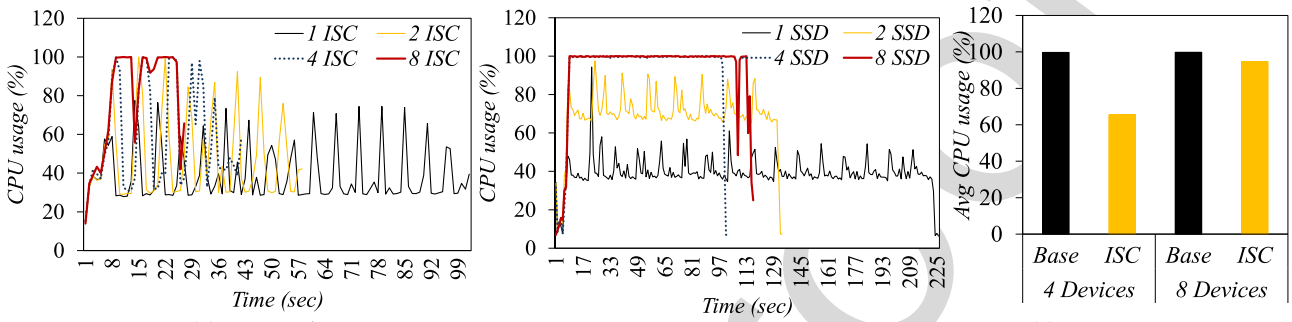
We set up two different Hadoop nodes with the same configuration: a commodity PC and a server. The commodity PC is a typical business PC equipped with an Intel i5-4570 processor (4 cores) and 8 GB RAM. The server is a more

IEEE Proof



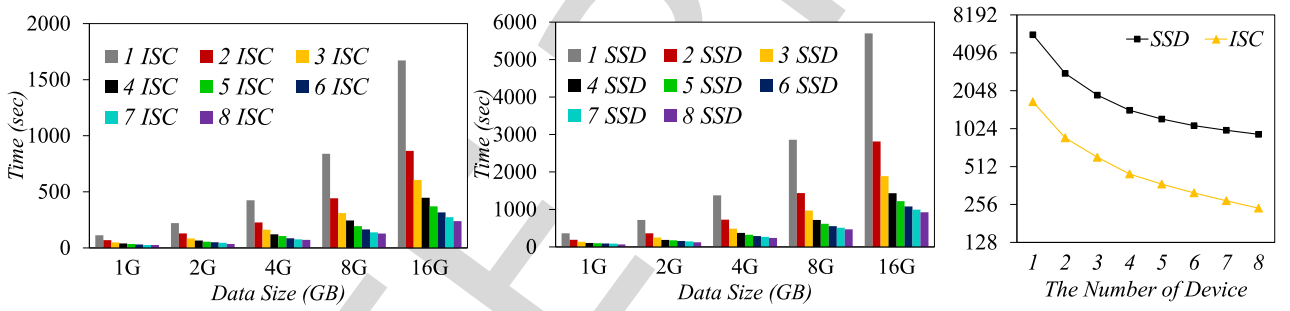
(a) ISC Hadoop (b) Typical Hadoop (c) Logarithmic Scale (16GB data)

Fig. 13. Hadoop system scalability (ISC versus Non ISC) on a desktop PC (Intel i5-4570 (4 cores)).



(a) ISC Hadoop (b) Typical Hadoop (c) Average CPU Usage

Fig. 14. Host CPU usage (ISC versus Non ISC) on a desktop PC (Intel i5-4570 (4 cores)).



(a) ISC Hadoop (b) Typical Hadoop (c) Logarithmic Scale (16GB data)

Fig. 15. Hadoop system scalability (ISC versus Non ISC) on a server (dual Intel Xeon E5-2630 (12 cores total)).

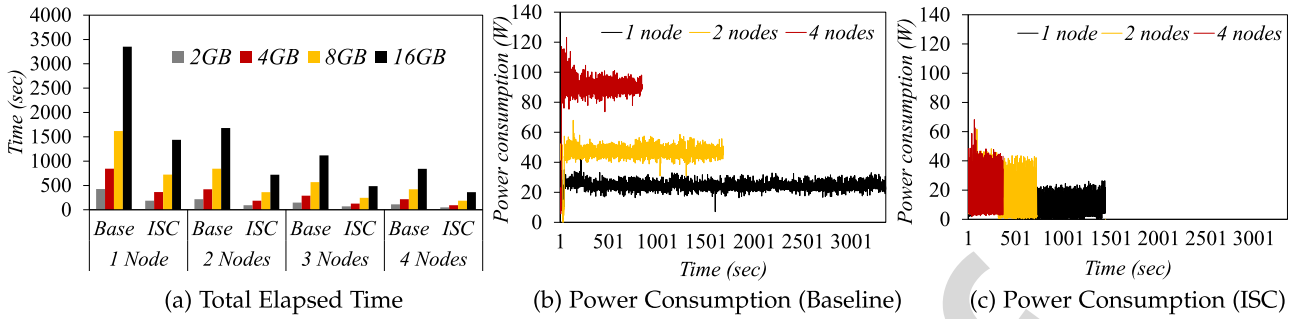
powerful system equipped with dual socket Intel Xeon E5-2630 (12 cores in total) and 32 GB RAM. We gradually increase the number of ISC devices (up to 8) installed in each single Hadoop node. Fig. 13 clearly presents the difference between the ISC system and the non-ISC system. As more devices are added, the non-ISC host system (typical Hadoop system) with four SSDs easily starts to saturate and the performance keeps degrading (Fig. 13b). On the other hand, our ISC Hadoop system performance keeps increasing until eight ISC devices (Fig. 13a) even in this low-cost commodity PC. Consequently, the performance gap increases from 2.3× to 6.2× (Fig. 13c). Fig. 14 plots host CPU usage of each Hadoop system with different numbers of devices and verifies these experimental results in the Fig. 13. We use 1 GB data for these experiments. As shown in the figure, the typical Hadoop system (Non-ISC) already saturates its host CPU with four devices (average 99.7 percent), while ISC Hadoop system only utilizes 65.5 percent of CPU performance with four ISC devices and reaches 94.7 percent of an average CPU utilization with eight ISC devices (Fig. 14c). Considering each system's total execution time, the ISC system can make a more significant contribution to a system scalability.

Now we make the same experiments in the server machine and, as we expected, find both systems scale well (Fig. 15). Note that since the base CPU clock of the sever (2.3 GHz) is lower than that of the commodity PC (3.2 GHz), the performance of the typical Hadoop system in the server is also relatively lower than the desktop PC (this performance gap is very close to the gap of CPU clocks). However, our ISC Hadoop system is significantly less sensitive to the host CPU performance (in this example, less than 8 percent performance gap. Fig. 13a versus Fig. 15a)

4.2.5 Hadoop Optimization

As stated in the Section 3.4, we tried to optimize Hadoop system performance and found three major bottleneck points. This section describes how much those optimizations contribute to performance improvement of each system. Fig. 17 shows a performance change after applying all optimizations. As shown in the Fig. 17a, our optimization

IEEE Proof



(a) Total Elapsed Time (b) Power Consumption (Baseline) (c) Power Consumption (ISC)

Fig. 16. Performance on Hadoop clusters.

improves an ISC Hadoop system by 40 percent and the typical Hadoop system by 27 percent so the performance gap increases from 2× to 2.3×. Fig. 17b breaks down each Hadoop task's execution time and shows the origination of the performance gain. The optimization affects the execution time of all tasks: (1) Both setup and cleanup times are equally reduced from 3 to 1 second (these are just one time tasks per job). (2) Each Reduce task execution time is also equally reduced from 2 to 1 second. (3) Most importantly, each Map task execution time decreases from 6 to 5 seconds for ISC Hadoop and from 15 to 13 seconds for typical Hadoop.

4.2.6 Hadoop Clusters

We set up five node clusters (one namenode and four datanodes) to verify whether all single node implications can be applied to real Hadoop clusters. Moreover, we need to verify our ISC Hadoop system seamlessly supports a distributed computing environment via a network connection. Each datanode is equipped with an ISC device and connects to each other via a 1 Gb Ethernet switch. We run the same Hadoop application and data, varying 1 to 16 GB (we eliminate results with 1 GB in the plots).

Fig. 16a presents the total elapsed time on the clusters. The results are very similar to those on a single node: our ISC Hadoop system is 2.3× faster than baseline Hadoop system, which naively implies that ISC Hadoop clusters of a half size can offer better performance than the baseline Hadoop clusters, or that ISC Hadoop clusters of the same size as the baseline Hadoop clusters can process 2.3× more amount of data for the same time period.

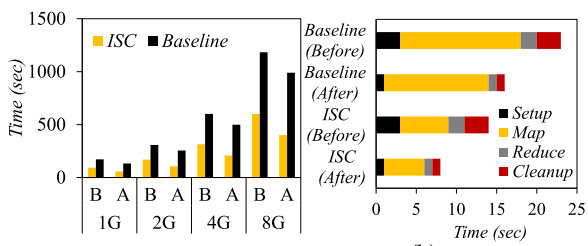
Figs. 16b and 16c present power consumption over time on each Hadoop cluster of varying size. We use 16 GB data and eliminate idle power. We also adopt an identical scale for both plots for convenient comparison. As displayed in the Fig. 16b, as the node size increases in typical Hadoop clusters, the power consumption also significantly increases in a steeper linear scale. On the other hand, the power consumption of our ISC Hadoop clusters gently increases so they consume even less power (up to 5× and 4.7× on average). As stated, considering the total execution time, a total energy consumption gap between two systems becomes significantly bigger by an average of 10.8×.

5 Related Work

In 1998, both Active Disks and iDisks started to explore modern In-Storage Computing (also called In-Storage Processing or Near-Data Processing) design [18], [19] with the help of the advance of HDD technologies (i.e., bandwidth growth and cost decrease). They tried to offload key application functions such as query operators inside HDDs to reduce data movement, corresponding to a current In-Storage Computing model. However, still-low HDD computing capabilities prevented HDD-based ISC from successful deploying in practical systems.

However, the advent of SSDs paved the way for people to reconsider In-Storage Computing a practical and promising computing model in industry as well as academia. Samsung recently promoted Storage Intelligence for In-Storage Computing and Multi-Stream technologies to improve SSD performance and increase NAND flash endurance [40], [41]. These are adopted by storage companies such as Pure Storage [42]. IBM applied ISC to their Blue Gene supercomputers to leverage the high SSD internal bandwidth [43]. Oracle's Exadata also started to offload complex database processing into their storage servers [44]. Recently, Samsung presented the possibility of applying ISC to a widely known embedded Key-Value store system (Google levelDB), thereby executing a data compaction task inside an SSD [45].

SSD-based ISC attracted academia as well. Logothetis et al. proposed iMR for in-situ log processing [46]. It suggested an architecture with a prototype system based on a best-effort distributed stream processor. Kim et al. tried to move a database scan operator to SSDs based on simulation [47]. Later, Do et al. integrated Microsoft SQL Server with real SSDs, thereby offloading database key operators to the SSDs [16]. Kang et al. proposed Hadoop MapReduce framework for ISC [25]. This work addressed the same topic as ours. However, it supported only one node with just limited functions. In contrast, our proposed ISC Hadoop MapReduce framework fully follows the existing MapReduce framework and operates on distributed Hadoop clusters. Boboila et al. proposed Active Flash to apply ISC to data analytics [48] and Tiwari et al. subsequently developed



(a) Total Elapsed Time (b) Breakdown

Fig. 17. Performance change after optimization. Here both A and B stand for *after* and *before* respectively.

IEEE Proof

more on Active Flash for extreme scale machines based on the OpenSSD platform [14], [49]. Seshadri et al. proposed a Willow system [50]. This was a user-programmable SSD prototype system allowing programmers to augment and extend SSD semantics with application-specific features. Recently, Wang et al. identified ISC opportunities in Web Search (Apache Lucene). This particularly addressed current ISC model limitations and demonstrated that highly CPU-intensive applications, such as decompression, cannot fully benefit from ISC computing model in terms of system performance (latency), but can still benefit from the ISC model in terms of energy consumption [17].

This work explores the opportunities and challenges of Hadoop MapReduce framework based on In-Storage Computing model.

6 Conclusion

This paper applied the SSD-based In-Storage Computing model to the Hadoop MapReduce framework—a de facto standard distributed computing framework for big data processing. After investigation, we offloaded the Mapper function from a host Hadoop system to SSDs and integrated the existing Hadoop MapReduce system with our ISC devices. This cooperative system design gave rise to the following challenging technical issues: (1) data representation discrepancy (2) system interfaces between a host system and an ISC device, (3) Hadoop data split handling, and (4) feature offloading. To address data representation and system interface issues, we added a software layer between the host Hadoop framework and our ISC devices. This software interface layer controlled file-to-LBA conversion and was in charge of the communication between host Java codes and device C/C++ codes. The 64 MB (by default) HDFS data split issue was another critical issue. To resolve this, we employed the host Hadoop system to receive the split information and store the split portion as a separate data file. Then, our ISC Hadoop system processed the separate file as new input data. Which feature(s) to offload to an ISC device was always a big issue for ISC research and implementation because moving all features to SSDs did not necessarily result in a performance improvement. A pseudo fully-distributed mode on a single Hadoop machine was yet-another interesting issue for Hadoop MapReduce research work. Although Hadoop did not support this mode, it could provide a very useful and efficient way to simulate fully-distributed Hadoop clusters on a single machine. All our initial studies and analyses have occurred in this mode. We studied Hadoop optimization as another important factor for both typical Hadoop system as well as ISC Hadoop system. The ISC Hadoop system could notably improve its performance by 40 percent only through optimization techniques.

Our extensive experiments and results demonstrated that our ISC Hadoop system exhibited a significant performance improvement ($2.3\times$ faster) as well as marvelous energy saving (about $11.5\times$ less) than a typical Hadoop system. They also verified that ISC was a promising computing model to effectively scale systems, thereby reducing host system resource consumption.

Acknowledgments

We would like to thank Dr. Kwanghyun Park and Prof. Jignesh M. Patel in Computer Science at the University of Wisconsin–Madison, USA for their valuable feedback. We also would like to thank David Schwaderer in Samsung Semiconductor Inc., USA for his valuable comments and proofreading.

References

[1] D. Park, B. Debnath, and D. Du, "CFTL: A convertible flash translation layer adaptive to data access patterns," in *Proc. ACM SIGMETRICS Int. Conf. Meas. Modeling Comput. Syst.*, 2010, pp. 365–366.

[2] A. Caulfield, L. Grupp, and S. Swanson, "Gordon: Using flash memory to build fast, power-efficient clusters for data-intensive applications," in *Proc. 14th Int. Conf. Archit. Support Programming Languages Operating Syst.*, 2009, pp. 217–228.

[3] D. Park, B. Debnath, and D. Du, "A dynamic switching flash translation layer based on a page-level mapping," *IEICE Trans. Inf. Syst.*, vol. E99-D, no. 6, pp. 51–60, Jun. 2016.

[4] Samsung, "Why SSDs are awesome: An SSD primer," Samsung Electronics, (2014). [Online]. Available: http://www.samsung.com/global/business/semiconductor/minisite/SSD/global/html/about/whitepaper01.html

[5] J. Wang, E. Lo, M. L. Yiu, J. Tong, G. Wang, and X. Liu, "The impact of solid state drive on search engine cache management," in *Proc. 36th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2013, pp. 693–702.

[6] R. Balasubramonian, et al., "Near-data processing: Insights from a MICRO-46 workshop," *IEEE Micro*, vol. 34, no. 4, pp. 36–42, Jul./Aug. 2014.

[7] D. Tiwari, S. S. Vazhkudai, Y. Kim, X. Ma, S. Boboila, and P. J. Desnoyers, "Reducing data movement costs using energy-efficient, active computation on SSD," in *Proc. USENIX Conf. Power-Aware Comput. Syst.*, 2012, pp. 4–4.

[8] Prabhat and Q. Koziol, *High Performance Parallel I/O*. Boca Raton, FL, USA: CRC Press, 2014.

[9] P. Rubens, "Big data and in-storage processing," (2013). [Online]. Available: http://www.enterprisestorageforum.com/storage-management/big-data-and-in-storage-processing.html

[10] Hortonworks, "What is Hadoop?" (2014). [Online]. Available: http://hortonworks.com/hadoop/mapreduce/

[11] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th Conf. Symp. Operating Syst. Des. Implementation*, 2004, pp. 10–10.

[12] T. White, *Hadoop: The Definitive Guide*, 3rd ed. Sebastopol, CA, USA: OReilly, 2012.

[13] J. Gray, "Distributed computing economics," Microsoft Res., Redmond, WA, USA, Tech. Rep. MSR-TR-2003-24, Mar. (2003). [Online]. Available: http://research.microsoft.com/apps/pubs/default.aspx?id=70001

[14] D. Tiwari, et al., "Active flash: Towards energy-efficient, in-situ data analytics on extreme-scale machines," in *Proc. 11th USENIX Conf. File Storage Technol.*, 2013, pp. 119–132.

[15] nVidia Corporations, "CUDA programming," (2012). [Online]. Available: http://www.nvidia.com/object/cuda_home_new.html

[16] J. Do, Y.-S. Kee, J. M. Patel, C. Park, K. Park, and D. J. DeWitt, "Query processing on smart SSDs: Opportunities and challenges," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 1221–1230.

[17] J. Wang, D. Park, Y. S. Kee, Y. Papakonstantinouy, and S. Swanson, "SSD in-storage computing for list intersection," in *Proc. 12th Int. Workshop Data Manage. New Hardware*, Jun. 2016, pp. 4:1–4:7.

[18] K. Keeton, D. A. Patterson, andJ. M. Hellerstein," A case for intelligent disks (IDISKs)," *ACM SIGMOD Rec.*, vol. 27, no. 3, pp. 42–52, 1998.

[19] A. Acharya, M. Uysal, and J. Saltz, "Active disks: Programming model, algorithms and evaluation," in *Proc. 8th Int. Conf. Archit. Support Programming Languages Operating Syst.*, 1998, pp. 81–91.

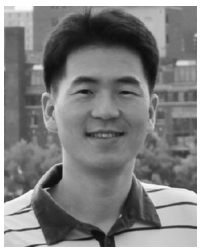
[20] A. De, M. Gokhale, R. Gupta, and S. Swanson, "Minerva: Accelerating data analysis in next-generation SSDs," in *Proc. IEEE 21st Annu. Int. Symp. Field-Programmable Custom Comput. Mach.*, 2013, pp. 9–16.

[21] T.-S. Chung, D.-J. Park, S. Park, D.-H. Lee, S.-W. Lee, and H.-J. Song, "A survey of flash translation layer," *J. Syst. Archit.*, vol. 55, no. 5/6, pp. 332–343, 2009.

[22] D. Park, B. Debnath, and D. Du, "A workload-aware adaptive hybrid flash translation layer with an efficient caching strategy," in *Proc. IEEE 19th Annu. Int. Symp. Modelling Anal. Simulation Comput. Telecommun. Syst.*, 2011, pp. 248–255.

IEEE Proof

[23] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google file system," in *Proc. 19th ACM Symp. Operating Syst. Principles*, 2003, pp. 29–43.
[24] J. Dean and S. Ghemawat, "MapReduce: A flexible data processing tool," *Commun. ACM*, vol. 53, pp. 72–77, 2010.
[25] Y. Kang, Y. suk Kee, E. L. Miller, and C. Park, "Enabling cost-effective data processing with smart SSD," in *Proc. IEEE 29th Symp. Mass Storage Syst. Technol.*, 2013, pp. 1–12.
[26] Seagate, "Seagate kinetic HDDS," (2014). [Online]. Available: http://www.seagate.com/products/enterprise-servers-storage/nearline-storage/kinetic-hdd/
[27] HGST, "Open Ethernet drive," (2014). [Online]. Available: http://www.hgst.com/science-of-storage/emerging-technologies/open-ethernet-drive-architecture
[28] N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, and R. Panigrahy, "Design tradeoffs for SSD performance," in *Proc. USENIX Annu. Tech. Conf.*, 2008, pp. 57–70.
[29] S. Lee and B. Moon, "Design of flash-based DBMS: An in-page logging approach," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2007, pp. 55–66.
[30] D. Park and D. Du, "Hot data identification for flash-based storage systems using multiple bloom filters," in *Proc. IEEE 27th Symp. Mass Storage Syst. Technol.*, 2011, pp. 1–11.
[31] D. Park, B. Debnath, Y. J. Nam, D. H. Du, Y. Kim, and Y. Kim, "An on-line hot data identification for flash-based storage using sampling mechanism," *ACM Appl. Comput. Rev.*, vol. 13, no. 1, pp. 51–64, 2013.
[32] D. Russell, "Tuning Hadoop on Dell PowerEdge Servers," Dell, Round Rock, TX, USA, 2013.
[33] Yokogawa, "Yokogawa WT300 series digital power meter," (2012). [Online]. Available: http://tmi.yokogawa.com/files/uploaded/WT300Series_Final.pdf
[34] F. Ahmad, S. Lee, M. Thottethodi, and T. N. Vijaykumar, "Puma: Purdue MapReduce benchmarks suite," Purdue Univ., Lafayette, IN, USA, Tech. Rep. TR-ECE-12-11, Oct. 2012.
[35] JDSU, "Xgig 1000 12 Gbps SAS analyzer," (2015). [Online]. Available: http://www.jdsu.com/ProductLiterature/xgig1000–12g-analyzer-ds-san-tm-ae.pdf
[36] A. N. Udipi, N. Muralimanohar, N. Chatterjee, R. Balasubramonian, A. Davis, and N. P. Jouppi, "Rethinking DRAM design and organization for energy-constrained multi-cores," in *Proc. 37th Annu. Int. Symp. Comput. Archit.*, 2010, pp. 175–186.
[37] K. Lim, P. Ranganathan, J. Chang, C. Patel, T. Mudge, and S. Reinhardt, "Understanding and designing new server architectures for emerging warehouse-computing environments," in *Proc. 35th Annu. Int. Symp. Comput. Archit.*, 2008, pp. 315–326.
[38] D. Meisner, B. T. Gold, and T. F. Wenisch, "PowerNap: Eliminating server idle power," in *Proc. 14th Int. Conf. Archit. Support Programming Languages Operating Syst.*, 2009, pp. 205–216.
[39] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, "Disaggregated memory for expansion and sharing in blade servers," in *Proc. 36th Annu. Int. Symp. Comput. Archit.*, 2009, pp. 267–278.
[40] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed solid-state drive," in *Proc. 6th USENIX Workshop Hot Topics Storage File Syst.*, 2014, pp. 13–13.
[41] Samsung, "Multi-Stream SSD Technology," Samsung Electronics, (2015). [Online]. Available: http://www.samsung.com/semiconductor/global/file/insight/2015/12/0_storage-intelligence-prodoverview-2015–0.pdf
[42] Pure Storage, "Pure storage flash array," (2015). [Online]. Available: http://www.purestorage.com
[43] IBM Jülich Research Center, "Blue gene active storage boosts I/O performance at JSC," (2013). [Online]. Available: http://cacm.acm.org/news/169841-blue-gene-active-storage-boosts-i-o-performance-at-jsc
[44] Oracle Corporation, "Oracle exadata white paper," 2010.
[45] Samsung, "In-storage computing: Ultimate solution for accelerating I/O intensive applications," Samsung Electronics, (2015). [Online]. Available: http://www.flashmemorysummit.com/English/Collaterals/Proceedings/2015/20150813_S301D_Ki.pdf
[46] D. Logothetis, C. Trezzo, K. C. Webb, and K. Yocum, "In-situ MapReduce for log processing," in *Proc. USENIX Conf. USENIX Annu. Tech. Conf.*, 2011, pp. 9–9.
[47] S. Kim, H. Oh, C. Park, S. Cho, and S. Lee, "Fast, energy efficient scan inside flash memory," in *Proc. Int. Workshop Accelerating Data Manage. Syst.*, 2011, pp. 36–43.
[48] S. Boboila, Y. Kim, S. S. Vazhkudai, P. Desnoyers, and G. M. Shipman, "Active flash: Out-of-core data analytics on flash storage," in *Proc. IEEE 28th Symp. Mass Storage Syst. Technol.*, 2012, pp. 1–12.
[49] OpenSSD project, "Jasmine openSSD platform," (2011). [Online]. Available: http://www.openssd-project.org/wiki/Jasmine_OpenSSD_Platform
[50] S. Seshadri, et al., "Willow: A user-programmable SSD," in *Proc. 11th USENIX Conf. Operating Syst. Des. Implementation*, 2014, pp. 67–80.



Dongchul Park received the PhD degree in computer science and engineering from the University of Minnesota Twin Cities, in 2012. He is currently a research scientist in Memory Solutions Laboratory (MSL), Samsung Semiconductor Inc., San Jose, California. He was a member of Center for Research in Intelligent Storage (CRIS) group under the advice of Professor David H.C. Du. His research interests focus on storage system design and applications including non-volatile memories (SSD), in-storage computing, Hadoop MapReduce, data center and cloud computing, data deduplication, key-value store, and shingled magnetic recording (SMR).



Jianguo Wang received the BS degree from Zhengzhou University, China, in 2009, and the MPhil degree in computer science from The Hong Kong Polytechnic University, in 2012. He is currently working toward the PhD degree at the University of California, San Diego. His research interests include data management system and new computing hardware.



Yang-Suk Kee received the BS, MS, and PhD degrees in electrical engineering and computer science from Seoul National University, South Korea. He is a director of Memory Solutions Laboratory (MSL), Samsung Semiconductor Inc., San Jose, California. He leads the Advanced Datacenter Solutions Group. Before joining Samsung, he worked for Oracle server technology group. Prior to his industrial experience, he worked on High Performance Distributed Computing and Cloud research in Information Sciences Institute, University of Southern California and Center of Networked Systems, University of California, San Diego.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

IEEE PROOF